

PriorAuthAI

Agentic AI Prior Authorization Assistant — Project Documentation

1. Project Overview

PriorAuthAI is an Agentic AI solution engineered to streamline and accelerate the healthcare prior authorization preparation and review process. Prior authorization is a health insurance requirement where healthcare providers must obtain advance approval from a health plan before performing specific procedures, tests, or prescribing medication.

The system ingests patient records, medical history, clinical documentation, and insurance guidelines to evaluate coverage readiness automatically. Using an agentic orchestration layer, PriorAuthAI systematically assesses whether all clinical criteria and documentation requirements are met for a requested service.

Important Note: PriorAuthAI does not make final, automated insurance determinations (i.e., it does not automatically issue official approvals or denials). Instead, it aggregates patient data, cross-references payer criteria, synthesizes clinical evidence, highlights documentation gaps, and generates a structured, evidence-backed review decision recommendation for human healthcare staff and claims reviewers.

2. Problem Statement

The current prior authorization process across the healthcare industry is heavily manual, slow, and error-prone, presenting significant operational and clinical challenges:

- **High Administrative Burden:** Clinical and administrative staff spend hours manually locating patient charts, policy documents, and payer-specific checklists.
- **Delayed Patient Care:** Manual paper-pushing delays necessary medical treatments, diagnostic tests, and surgeries.
- **High Denials Due to Missing Data:** A significant percentage of prior authorization requests are rejected due to simple errors, such as missing clinical notes, absent diagnostic reports, or mismatched insurance policy codes.
- **Repetitive Queries:** Administrative teams frequently spend time re-keying patient details or re-querying internal systems to verify basic patient information during case reviews.

3. Main Objectives

- **Automate Information Retrieval:** Use standard tool interfaces to pull patient profiles, clinical history, diagnostic notes, and coverage requirements automatically.
- **Accelerate Case Review Timelines:** Reduce review prep time from hours to minutes by automating document completeness checks and clinical evidence extraction.

- **Identify Missing Clinical Evidence:** Flag missing documentation (e.g., MRI reports, physical therapy logs, physician letters) before formal submission to minimize administrative rejections.
- **Deliver Context-Aware Conversational Support:** Provide a chat interface that maintains session memory for the active patient, eliminating repetitive data entry.
- **Provide Objective Review Recommendations:** Present structured, transparent, evidence-based recommendations to human reviewers without replacing medical oversight.

4. Key Features

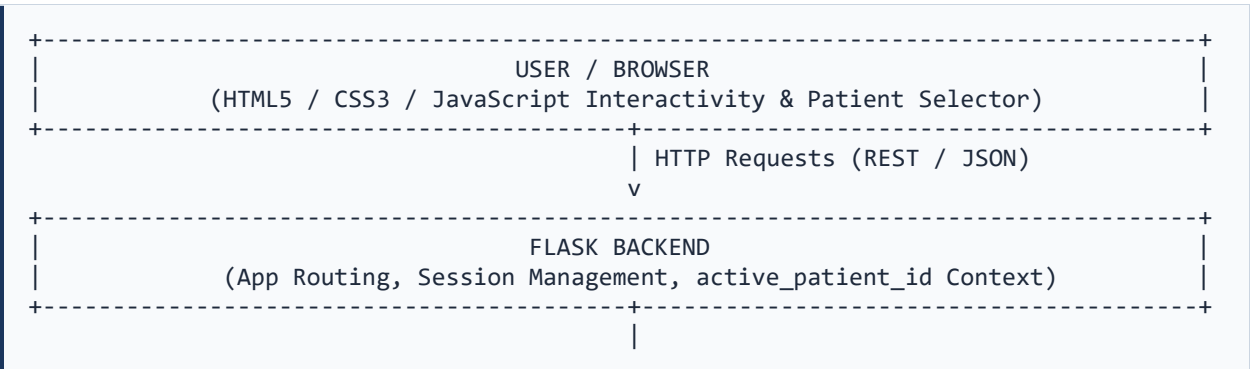
- **Patient-Centric Session State:** Automatically locks to an active patient ID across both the automated workflow and the interactive chatbot.
- **Automated Document Completeness Verification:** Checks whether required clinical artifacts (e.g., lab results, imaging, chart notes) exist in the record.
- **Clinical Evidence Analysis:** Extracts key diagnostic indicators, treatment history, and clinical observations from stored clinical documents.
- **Coverage Requirements Evaluation:** Evaluates procedure requests against predefined payer criteria and insurance policy rules.
- **Structured Review Preparation:** Generates a summarized case summary, missing items checklist, and a preliminary review decision recommendation.
- **Interactive AI Assistant:** Enables healthcare staff to query patient history, insurance details, authorization status, and missing documents in natural language.
- **Decoupled Tool Protocol Integration:** Uses Model Context Protocol (MCP) to decouple tool interfaces from backend application logic.

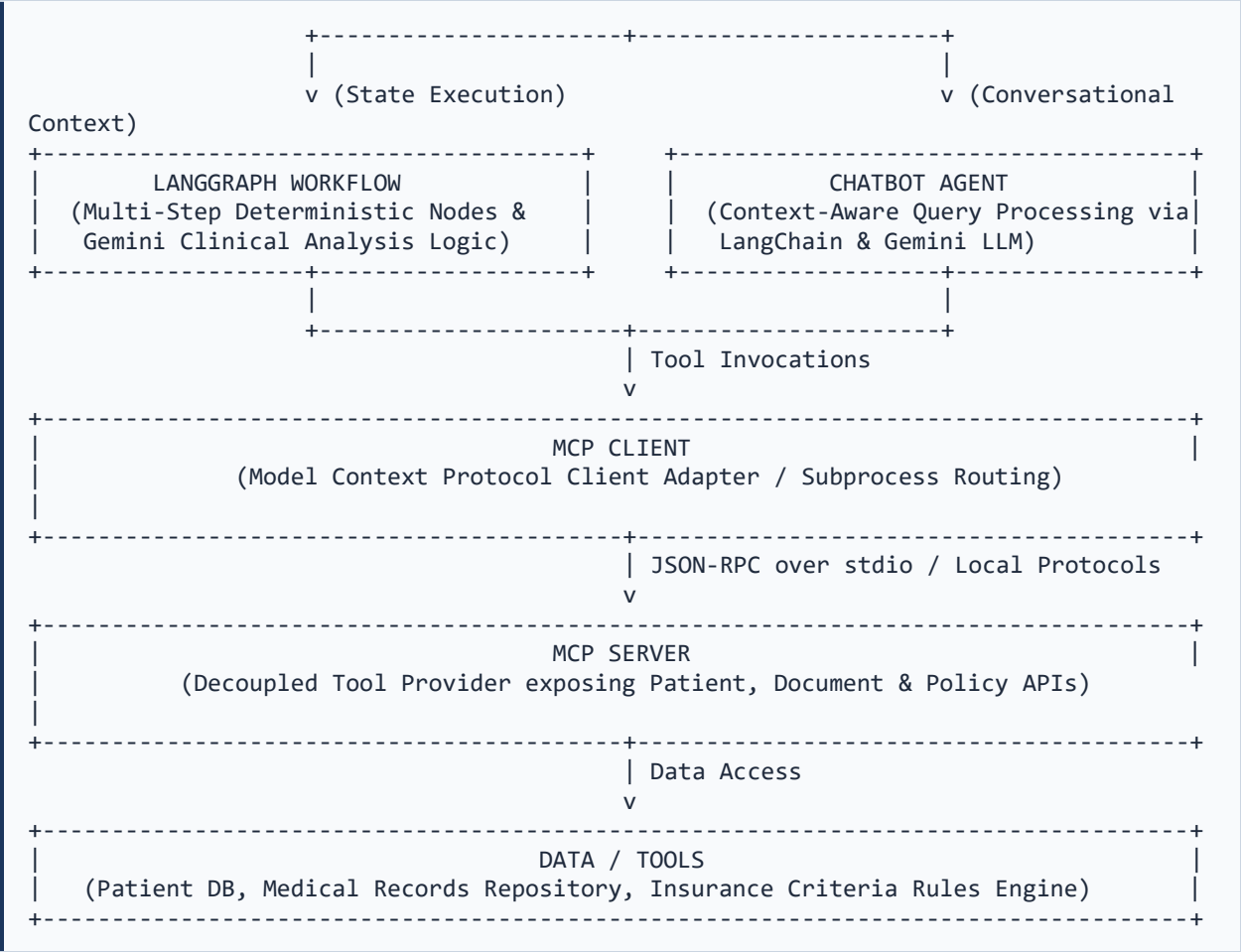
5. System Architecture

The system processes user requests through a multi-tier flow, connecting client requests through an orchestration pipeline down to isolated tools and data providers:

Execution Flow: User → Frontend UI → Flask Backend → LangGraph Agentic Workflow → MCP Client → MCP Server → Data / Tools → Response

Architecture Diagram





6. Model Context Protocol (MCP) Integration

Model Context Protocol (MCP) serves as the standardized communication bridge between the system's reasoning components (LangGraph/Gemini) and the data sources. By separating tool execution from the application core, MCP ensures tool calls follow a strict JSON-RPC standard.

Implemented MCP Tools

Tool Name	Input Parameters	Output / Action Description
get_patient_details	patient_id (str)	Fetches demographics, age, policy details, primary diagnosis, and requested CPT procedure code.
get_patient_documents	patient_id (str)	Returns metadata list of uploaded clinical documents (MRI report, physician notes, PT logs).
read_document_content	patient_id (str), document_type (str)	Retrieves raw text content of a specific clinical document for LLM analysis.
get_authorization_rules	procedure_code (str)	Returns mandatory payer requirements, clinical criteria,

and required supporting file list.

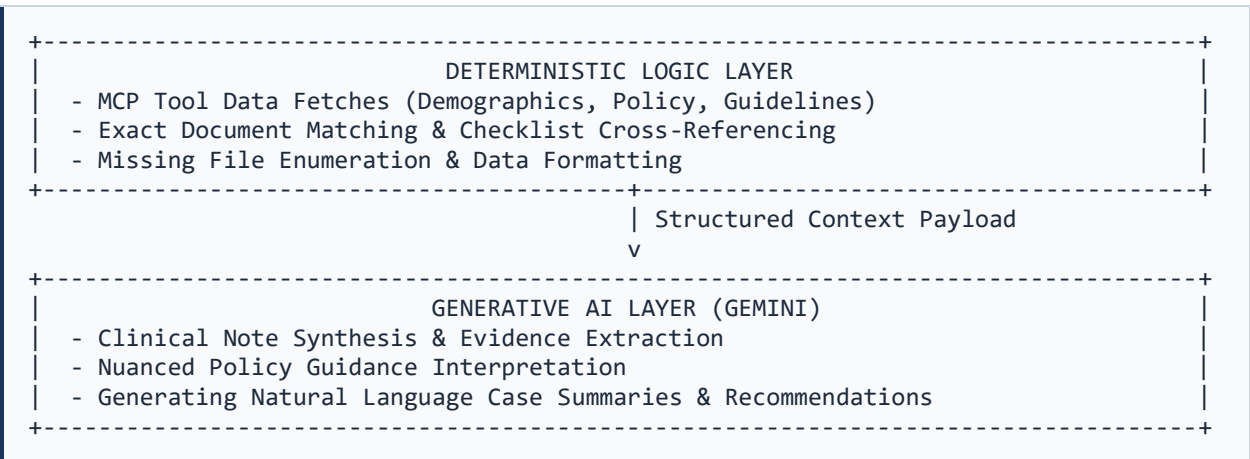
check_document_status

patient_id (str),
procedure_code (str)

Cross-references uploaded documents against rules to identify present vs missing files.

7. Agentic AI & LangGraph Workflow

To prevent unpredictable or 'hallucinated' execution flows, PriorAuthAI cleanly separates deterministic business rules from generative language modeling.



LangGraph Execution Steps

- **1. State Node Execution:** Nodes run sequentially, updating a shared dictionary (PriorAuthState) that holds current patient data, fetched documents, identified gaps, and analysis output.
- **2. Deterministic Data Gathering:** Early nodes execute MCP tools directly to obtain verified data (patient record, required rules, available files).
- **3. LLM Reasoning Nodes:** Once data is collected, specialized prompt nodes invoke Google Gemini to analyze clinical notes, extract evidence matching payer guidelines, and draft clinical rationales.
- **4. Structured Decision Node:** The final node aggregates deterministic checklist results and LLM clinical summaries to generate a structured case review package.

8. Chatbot Architecture & Context Persistence

The interactive chatbot allows healthcare staff to inquire about patient cases naturally without re-entering administrative context on every turn.

- **Session-Bound Patient Lock:** When a user selects a patient in the UI, Flask stores the selection in the session state (active_patient_id).
- **Automatic System Prompt Injection:** Every prompt submitted to the chatbot is appended with system context: System Context = { Active Patient ID: 'PT-8821' }.

- **Targeted Tool Calls:** When users ask general questions such as 'Does this patient meet the requirements?' or 'Show me their diagnosis', the chatbot passes `active_patient_id` to the underlying MCP tools automatically.

9. Detailed Workflow Breakdown

- **Phase 1: Load Patient & Policy Details** — The workflow fetches core demographics, primary diagnosis, requested procedure (CPT/HCPCS code), and corresponding health plan authorization guidelines via MCP tools.
- **Phase 2: Document Completeness Verification** — The system queries the record for uploaded artifacts and compares them against required files. Missing items are added to a `missing_documents` list.
- **Phase 3: Clinical Evidence Extraction** — If required documents are present, the system reads their text and uses Gemini to extract key diagnostic metrics, conservative treatment history, and physical evaluation notes.
- **Phase 4: Policy Requirements Evaluation** — Extracted evidence is cross-referenced against policy rules. The system marks each requirement as Met, Unmet, or Insufficient Evidence.
- **Phase 5: Synthesis & Decision Recommendation** — The workflow compiles findings into a structured summary report containing Case Summary, Document Status Checklist, Requirement Audit, and Review Decision Recommendation.

10. Technology Stack

Component Layer	Technology	Purpose / Application
Backend Framework	Python / Flask	Web server, session management, REST API endpoints.
Agentic Framework	LangGraph / LangChain	Multi-step stateful workflow orchestration & chain execution.
Generative AI Model	Google Gemini	Clinical text processing, evidence extraction, response generation.
Tool Protocol Interface	Model Context Protocol (MCP)	Decoupled client-server framework for data tools.
Frontend Interface	HTML5, CSS3, JavaScript	Lightweight web dashboard and chat UI.
Execution Protocol	JSON-RPC over stdio	Transport layer between MCP client and server process.

11. Testing & Quality Assurance

The project maintains a comprehensive test suite covering tool integrations, workflow execution, context persistence, and rule logic:

```
===== TEST EXECUTION SUMMARY =====  
Directory: ./tests/  
Total Test Cases: 23  
Passing Tests:    23  
Failing Tests:    0  
Status:          100% PASS RATE  
=====
```

- **MCP Tool Tests:** Validates proper request formatting, input parsing, and error handling across all 5 MCP tools.
- **Workflow Tests:** Verifies state transitions in the LangGraph orchestration pipeline from initial data load through final summary.
- **Session Tests:** Confirms active_patient_id remains stable across HTTP requests and correctly updates chatbot context.

12. Example Use Case Scenario

Scenario: User selects Patient PT-1042 (John Doe) requesting CPT 73721 (Knee MRI).

User Prompt: "Does this patient have all the required documents uploaded for their MRI?"

System Action: Retrieves active_patient_id = 'PT-1042'. Calls check_document_status(patient_id='PT-1042', procedure_code='73721').

PriorAuthAI Response: For Patient PT-1042 (John Doe), the requested procedure CPT 73721 (Knee MRI) requires 3 key documents:

1. Physician Clinical Notes - Uploaded (Received 08/12/2026)
2. X-Ray Report - Uploaded (Received 08/14/2026)
3. Physical Therapy Log (6 Weeks) - MISSING

Current Status: Incomplete Documentation. The 6-week conservative physical therapy trial record is required before medical review.

13. Current MVP Scope

- **Supported Domain:** Focused on outpatient diagnostic imaging (e.g., Knee/Lumbar MRI) and elective surgical procedures.
- **Deployment Model:** Runs locally via Flask with MCP subprocess transport via standard IO.
- **Data Storage:** JSON-based mock medical records database and rule engine providing real-time data access.

14. Future Improvements

- **EHR Integration:** Direct integration with hospital EHRs (Epic, Cerner) via FHIR APIs.

- **OCR Document Ingestion:** Document parsing pipeline for handwritten clinical notes and faxed PDFs using OCR.
- **Granular RBAC:** Role-based view customization for provider staff vs payer medical reviewers.

15. Conclusion

PriorAuthAI demonstrates how Agentic AI can streamline a complex administrative workflow in healthcare. By combining LangGraph orchestration, Google Gemini generative capabilities, and Model Context Protocol (MCP) tool isolation, the system transforms unstructured patient data into organized, actionable review packages.

By handling data collection, cross-referencing policy guidelines, and tracking document completeness automatically, PriorAuthAI reduces administrative overhead and minimizes submission errors—ensuring human clinicians can deliver timely care.

16. Observability, Tracing & Monitoring

An observability layer was added to PriorAuthAI to make the agent's execution measurable, traceable, and easier to debug. Instead of treating the agent as a black box, the system records workflow and chatbot activity and presents it through a Streamlit dashboard.

Key Implementation Changes

Area	Implementation
Distributed-style execution tracing	Workflow, tool, LLM, error, retry, and evaluation events are recorded as structured trace events in JSONL format.
Real-time Streamlit dashboard	Added a monitoring dashboard for workflow health, reliability, latency, cost, evaluation results, trends, and trace inspection.
Workflow & tool metrics	Tracks task completion, escalation, tool success, user flags, invalid schema rate, retries, fallback calls, P50/P95 latency, and cost per successful case.
LLM quality evaluation	Added nine evaluation signals: Hallucination Rate, Faithfulness, System Prompt Adherence, Goal Achievement, Tool Selection Accuracy, Tool Usage Efficiency, Decision Accuracy, Escalation Accuracy, and Guardrail Violation Score.
Guardrails & auditability	Guardrail checks and evaluation results are recorded alongside trace data so failures and unsafe decision patterns can be investigated.
Automatic refresh	The dashboard can automatically refresh every 5 seconds and also supports manual refresh, allowing newly generated trace events to appear without restarting the dashboard.
Reliability & deployment fixes	Added the MCP dependency, replaced the machine-specific MCP subprocess path with the active Python interpreter, and fixed the dashboard event-loading issue.
Provider flexibility	LLM configuration was made provider/model configurable, enabling OpenRouter to be used when the Gemini service is unavailable or overloaded.

Why Observability Matters

The observability layer provides visibility into how the agent behaves in production-like runs: what tools were called, how long operations took, whether workflows completed or escalated, where failures occurred, and how the generated responses performed against quality and guardrail checks. This supports debugging, performance monitoring, quality evaluation, and auditability.

Data source: dashboard metrics are calculated from recorded trace/evaluation events, so the displayed values reflect actual execution data.

Current limitation: a document-type naming mismatch remains in some sample data and can affect task-completion/escalation results. The monitoring layer makes this behavior visible for diagnosis rather than masking it.